Hamleys Group Detailed Presentation Guide

Team 05 | 19-slide speaking notes and viva preparation

Purpose of this guide

This document contains detailed but presentation-friendly explanations. The boxed paragraphs can be spoken directly, while the supporting notes help you understand the meaning and answer follow-up questions.

- Start each slide with its purpose, then explain the automation work and end with the benefit or result.
- Do not read too quickly. Pause after the framework flow, defect and project-challenge slides.
- Use the exact project terms: Hamleys India, Playwright, TypeScript, Hybrid Framework, BDD Framework, Page Object Model and UI-store.

Slide 1: Project Introduction

Suggested explanation

Good morning everyone. Our project is the Playwright Hamleys India Automation Capstone, completed under Project Gladiator. The objective was to automate important customer-facing journeys on the Hamleys India website using Playwright and TypeScript. We implemented the project in two formats: a Hybrid Framework for structured technical automation and a Cucumber BDD Framework for readable business scenarios.

What this slide means

This slide establishes the project identity, application and two framework approaches. The group presentation covers the complete team scope rather than one member's module.

Points to emphasize

- Mention Playwright, TypeScript, Hybrid and BDD.
- Use Hamleys India or hamleys.in while speaking.

Transition: I will first outline the sections included in our presentation.

Slide 2: Agenda

Suggested explanation

Our presentation is divided into five areas. First, we introduce the application, project scope and outcomes. Second, we explain the software and hardware environment. Third, we cover automation features, utilities and reusable components. Fourth, we describe the Hybrid and BDD frameworks with their architecture and folder structures. Finally, we show execution evidence, challenges, team contribution and learnings.

What this slide means

The agenda gives the audience a clear sequence from business scope to technical design and project outcome.

Points to emphasize

- Use this as a roadmap only.

Transition: Let us begin with the Hamleys India application.

Slide 3: Application Overview

Suggested explanation

Hamleys India is an e-commerce website for toys, games, gifts and children's products. Customers can discover products through menus, collections, brands, age groups, search, sorting and filters. Customers can also view product details, manage wishlist and bag actions, and use supporting links. Our testing focused on navigation, functionality, messages, product information, cart actions and complete customer journeys.

What this slide means

The user journey can be described as discover, evaluate, act and confirm. The automation checks that each stage provides the expected information or result.

Points to emphasize

- Use Hamleys India, not the unrelated internal HealthKart heading.
- Connect website functions with customer goals.

Transition: Based on this application, the team selected the following modules.

Slide 4: Modules

Suggested explanation

The overall scope covered navigation, Sale, Return Gifts, Search, Filters, Product Details, Wishlist and Cart. Within these areas, the team tested menu navigation, collections, brands, age-based discovery, category and price filters, product information, favourites, bag actions, login, OTP, privacy-related pages, newsletter behaviour and selected footer journeys.

What this slide means

The module list demonstrates broad functional coverage across product discovery, customer accounts, shopping actions and supporting content.

Points to emphasize

- Mention representative examples instead of reading every item.
- State that the direction included functional, positive, negative and regression testing.

Transition: The next slide groups this scope into the main application journeys tested.

Slide 5: Application Modules Tested

Suggested explanation

The Sale module covered browsing by price or category, product listings, filters and cart actions. Return Gifts included Under 500, Shop All, product details, wishlist and collection-route validation. Shop Toys covered category navigation, filters, product selection, details and bag actions. Toys by Age covered age-based collections, sorting and product validation. Brand and Search included journeys for selected brands. Login, Wishlist and Cart covered mobile number, OTP, customer details, pincode, coupon, bag and checkout-related validations.

What this slide means

This slide translates high-level modules into actual functional activities. It also shows that each module contains navigation, interaction and validation rather than a single click.

Points to emphasize

- Keep the explanation grouped by module.
- Do not read every brand unless asked.

Transition: After executing these journeys, the project produced the following testing outcomes.

Slide 6: Testing Outcomes

Suggested explanation

The team validated key journeys across Sale, Return Gifts, Shop Toys, Toys by Age, Brands, Login, Wishlist and Cart. We checked results using page headings, URLs, visible elements, product information, messages and cart state. The framework produced reusable page objects, UI-store locators, utilities, logs, screenshots and reports. Traceability was maintained through test files, feature files, step definitions, page modules and execution evidence.

What this slide means

The outcome is not only a set of executed tests. The project also created reusable automation assets that can support repeated regression execution and easier failure investigation.

Points to emphasize

- Mention functional confidence and reusable foundation.
- Connect every result with evidence.

Transition: The next section explains the environment required to execute the project.

Slide 7: Software Requirements

Suggested explanation

The automation was developed and executed using Windows 11 and Google Chrome. Playwright was the automation tool, and TypeScript was the programming language. HTML reports, logs and screenshots were used for results and evidence. Excel and environment configuration supported test data and settings. Git and GitHub supported version control and team collaboration.

What this slide means

Each software component has a separate purpose: development, execution, data management, evidence or collaboration. Together they form the working project environment.

Points to emphasize

- Explain purpose, not only product names.
- Differentiate automation tool, language and version-control system.

Transition: In addition to software, stable hardware is needed for browser automation.

Slide 8: Hardware Requirements

Suggested explanation

The framework can be executed on a suitable desktop or laptop with a dual-core processor or higher, at least 8 GB of memory, sufficient storage for dependencies and evidence, a stable internet connection and a standard desktop display. These requirements support local browser execution and reduce interruptions caused by slow loading, limited memory or insufficient storage.

What this slide means

Browser automation consumes resources for the browser, Node.js process, reports, logs and screenshots. A stable connection is also important because Hamleys is a live external website.

Points to emphasize

- Present these as project environment requirements, not universal Playwright limits.

Transition: With the environment established, I will explain the automation capabilities used in the project.

Slide 9: Automation Features

Suggested explanation

The framework automates common customer actions including hover, click, fill, clear, scroll, sort, filter and browser navigation. It validates product title, image, price, description, specifications and availability where included in the scenario. Shopping actions include wishlist, favourites, Add to Bag, pincode, coupon and checkout flows. Navigation handling includes menus, footer links, browser back, URL validation and new-tab handling. Assertions, logs, screenshots and reports provide evidence for each execution.

What this slide means

This slide shows that automation includes both interactions and validations. A script is only complete when the expected result is asserted and recorded.

Points to emphasize

- Group the features into user actions, validations, shopping actions and evidence.

Transition: Common technical operations were moved into reusable utilities.

Slide 10: Utilities

Suggested explanation

ExcelReader reads test data for data-driven execution. ConfigReader manages values such as the base URL, browser settings, paths and environment values. Logger records actions, validations and errors. ScreenshotUtil captures visual evidence at important checkpoints or failures. Report utilities summarize execution results. Browser helpers provide common navigation and waiting behaviour. Page objects contain reusable application actions, while the UI-store keeps locators separate from the test logic.

What this slide means

Utilities improve consistency and reduce duplicate code. Centralizing common technical operations also makes future updates easier because the change can be applied in one place.

Points to emphasize

- Explain one responsibility per utility.
- Keep business flow in tests or features, not in generic utilities.

Transition: These components are organized into our Hybrid Framework.

Slide 11: Hybrid Framework

Suggested explanation

A Hybrid Automation Framework combines several design approaches. In this project, it combines Page Object Model, data-driven testing, reusable utilities, configuration, logging and reporting. The test layer contains business flows, the page-object layer contains reusable actions and validations, the utility layer handles common technical functions, the UI-store stores selectors and literals, and the data and configuration layer provides external values. Playwright performs browser execution and assertions.

What this slide means

The main benefit is separation of concerns. A change in a locator, input value or report method does not require rewriting the complete test scenario.

Points to emphasize

- Explain the purpose of each layer.
- Mention maintainability, reusability and readability.

Transition: The architecture diagram shows how these layers communicate during execution.

Slide 12: Architecture of Hybrid Framework

Suggested explanation

Execution begins when a Playwright test file starts a business journey. The test obtains data and configuration from Excel or environment files where required. It calls a method from the appropriate page object. The page object retrieves the locator from the UI-store and performs the action through Playwright. Assertions verify the expected result. Logger, screenshot, wait and report utilities run around these actions to create diagnostic information and final evidence.

What this slide means

The architecture provides a clear path from test intent to browser action. It prevents the test file from becoming overloaded with selectors, repeated actions and reporting code.

Points to emphasize

- Flow: Test -> Data/Config -> Page Object -> UI-store -> Playwright -> Assertion -> Report.

Transition: The project also implements the same business journeys using BDD.

Slide 13: BDD Framework

Suggested explanation

BDD stands for Behavior-Driven Development. In the BDD framework, expected application behaviour is written in a feature file using Given, When and Then. Feature files remain readable, while TypeScript step definitions connect each Gherkin line to the automation implementation. Page objects perform reusable actions, the UI-store provides centralized selectors, hooks manage browser setup and cleanup, and utilities provide logs, screenshots, waits and execution evidence.

What this slide means

BDD makes requirements and scenarios easier to understand while preserving a structured technical implementation behind the steps.

Points to emphasize

- Given is context, When is action, Then is expected result.
- Keep step definitions short and delegate reusable work to page objects.

Transition: The next slide shows the full BDD execution path.

Slide 14: Architecture of BDD Framework

Suggested explanation

Cucumber first reads the feature file and matches every Given, When and Then step with a step definition. Before the scenario, a hook creates the required browser and page context. The step definition calls a reusable page-object method, and the page object uses the UI-store locator. Playwright performs the browser interaction and assertion. During execution, utilities record logs and screenshots. After the scenario, another hook captures the final evidence where required and closes the browser resources.

What this slide means

The architecture separates business language, implementation logic, element selectors and lifecycle management. This makes scenarios easier to read and code easier to maintain.

Points to emphasize

- Flow: Feature -> Step Definition -> Page Object -> UI-store -> Playwright -> Report.
- Hooks should manage lifecycle rather than business steps.

Transition: The following slides show how these layers are represented in the repository.

Slide 15: Hybrid Framework Folder Structure

Suggested explanation

The tests folder contains the separate Playwright test files that start each business flow. The pages folder contains reusable page classes. The UI-store folder contains page locators and literals. The utilities folder contains logger, screenshot, report and helper code. The test-data folder contains external data such as Excel input. The environment file contains configuration values, while playwright.config.ts controls Playwright execution settings.

What this slide means

A clear folder structure helps team members quickly find the correct file and reduces accidental duplication or mixing of responsibilities.

Points to emphasize

- Explain folder responsibility, not only the folder name.

Transition: The BDD folder structure adds feature, step-definition and hook layers.

Slide 16: BDD Framework Folder Structure

Suggested explanation

The features folder contains ten readable feature files, and the step-definitions folder contains the corresponding TypeScript implementation files. The pages and UI-store folders provide reusable actions and locators. The utilities folder contains common support code. The hooks folder manages browser setup, scenario context and cleanup. The reports folder stores execution evidence and summaries.

What this slide means

The BDD structure separates business-readable scenarios from implementation details while still reusing page objects, locators and utilities.

Points to emphasize

- Feature and step files should map clearly.
- Shared hooks avoid repeated browser initialization in every step.

Transition: After execution, the framework generates report evidence.

Slide 17: Execution Report

Suggested explanation

The execution report provides a consolidated view of the automation run. It shows the executed test or scenario names, passed and failed status, step-level logs, failure details, screenshots or attachments, execution timing and the overall summary. The report helps the team identify where a failure occurred and provides evidence that can be reviewed without rerunning the scenario immediately.

What this slide means

Reporting improves debugging, traceability and communication. A useful report should show what ran, what failed and what evidence is available.

Points to emphasize

- Call it the automation execution report unless the exact report technology is confirmed.
- Explain how screenshots and logs support root-cause analysis.

Transition: While building and executing the framework, the team handled several practical challenges.

Slide 18: Project Challenges

Suggested explanation

The first challenge was dynamic elements and hover menus, because visibility could change while the test was running. We handled this with role-based or text-based locators and condition-based waits. The second challenge was changing product inventory and listing order on the live website. We used product keywords, availability checks and reusable product-selection methods. The third challenge was multiple matching elements, which caused strict-mode issues. We resolved this by using exact, visible and section-scoped locators.

What this slide means

The key presentation structure is issue, cause and resolution. These challenges demonstrate why stable automation requires more than simply recording clicks.

Points to emphasize

- Avoid fixed delays where a condition can be awaited.
- Scope locators to the intended section.
- Treat changing inventory as dynamic test data.

Transition: I will conclude with the team’s work distribution and final outcome.

Slide 19: Team Contribution and Conclusion

Suggested explanation

The work was distributed among five team members, with each member responsible for two flows and contributions to page classes, utilities and the UI-store. My responsibility included Flows 1 and 10 along with related framework components. As a team, we created reusable Hybrid and BDD automation structures for important Hamleys customer journeys, supported by external data, centralized locators, logs, screenshots and reports. Thank you, and we are ready to answer questions.

What this slide means

The final slide recognizes shared contribution while clearly stating personal ownership. It should close the presentation by summarizing both functional coverage and framework quality.

Points to emphasize

- Keep other members’ responsibilities brief unless asked.
- State personal contribution clearly.

Transition: Thank you.

Detailed Viva Questions and Answers

What is Playwright?

Playwright is a web automation framework that controls browsers and validates modern web applications. In the Hamleys project, Playwright was used with TypeScript to perform navigation, user actions and assertions, and to support screenshots, tracing, tabs and reporting.

What are the main features of Playwright?

Important features include support for Chromium, Firefox and WebKit; automatic waiting for actionable elements; role, text, label and test-ID locators; browser-context isolation; parallel execution; multiple tabs and pop-ups; built-in assertions; screenshots; video and trace support; network monitoring; and device emulation.

How is Playwright different from Selenium?

Both are used for browser automation. Selenium is mature and has a broad ecosystem. Playwright provides modern capabilities such as auto-waiting, browser contexts, tracing, network handling and event-based page handling closely integrated into the framework. The best choice depends on application needs, existing skills and project standards.

What is a Hybrid Framework?

A Hybrid Framework combines multiple design approaches. Our framework combines Page Object Model, data-driven testing, centralized locators, reusable utilities, configuration, logging, screenshots and reporting.

Why is Page Object Model used?

Page Object Model stores reusable page actions and validations in page classes. Tests call meaningful methods rather than repeating low-level browser actions. This reduces duplication, improves readability and allows page changes to be maintained centrally.

What is the purpose of the UI-store?

The UI-store keeps element locators and selected page literals separate from tests and page logic. If a selector changes, the locator can be updated centrally instead of changing many test files.

How does the Hybrid Framework work?

The test file starts the business scenario and reads data or configuration where required. It calls a page-object method, which uses a UI-store locator. Playwright interacts with the browser and assertions verify the expected result. Utilities capture logs, screenshots and report details.

How does the BDD Framework work?

The feature file defines behaviour using Given, When and Then. Cucumber maps each line to a step definition. Step definitions call page-object methods, page objects use UI-store locators, hooks manage browser setup and cleanup, and utilities capture execution evidence.

What is the difference between Hybrid and BDD in this project?

The Hybrid tests describe flows directly in Playwright test files, while BDD describes behaviour first in feature files and connects it to code through step definitions. Both can reuse page objects, locators and utilities.

Why are hooks used in BDD?

Hooks centralize activities that happen before or after scenarios, such as creating the browser context, providing the page to steps, capturing failure evidence and cleaning up browser resources.

Why did you use condition-based waits?

A condition-based wait continues when the actual required state is reached, such as an element becoming visible or a page reaching the expected URL. A fixed delay may be unnecessarily long or still too short when the website is slow.

How did you solve strict-mode locator errors?

We refined the locator so that it matched the intended element only. This included using exact text or role, checking visibility and scoping the locator to the correct navigation, filter or footer section.

How did you handle changing products on a live website?

We avoided depending only on a fixed product position. Product-selection logic used meaningful keywords or visible-product checks and validated availability before performing actions that depended on stock.

What happens when a scenario fails?

The assertion or action reports the failure, the logger records the error, and the framework captures available evidence such as a screenshot and report details. This information helps identify whether the cause is a locator, timing, data or application issue.